ЛАБОРАТОРНА РОБОТА № 3

REST API з Express.js та інтеграційне тестування

Мета: Розробити REST API на Node.js і TypeScript із застосуванням Express.js, Zod та in-memory сховища. Навчитися розділяти конфігурацію застосунку й запуск сервера, обробляти помилки через middleware, тестувати HTTP-маршрути за допомогою Supertest та аналізувати покриття.

Хід роботи

1 Перевірка середовища та залежностей

Перевірено встановлені Node.js та npm і перелік прямих залежностей проєкту. Команда npm ls --depth=0 показує пакети, які безпосередньо зазначені в package.json. Для повторюваного встановлення залежностей створено package-lock.json.

```
node --version
npm.cmd --version
npm.cmd ls --depth=0
```

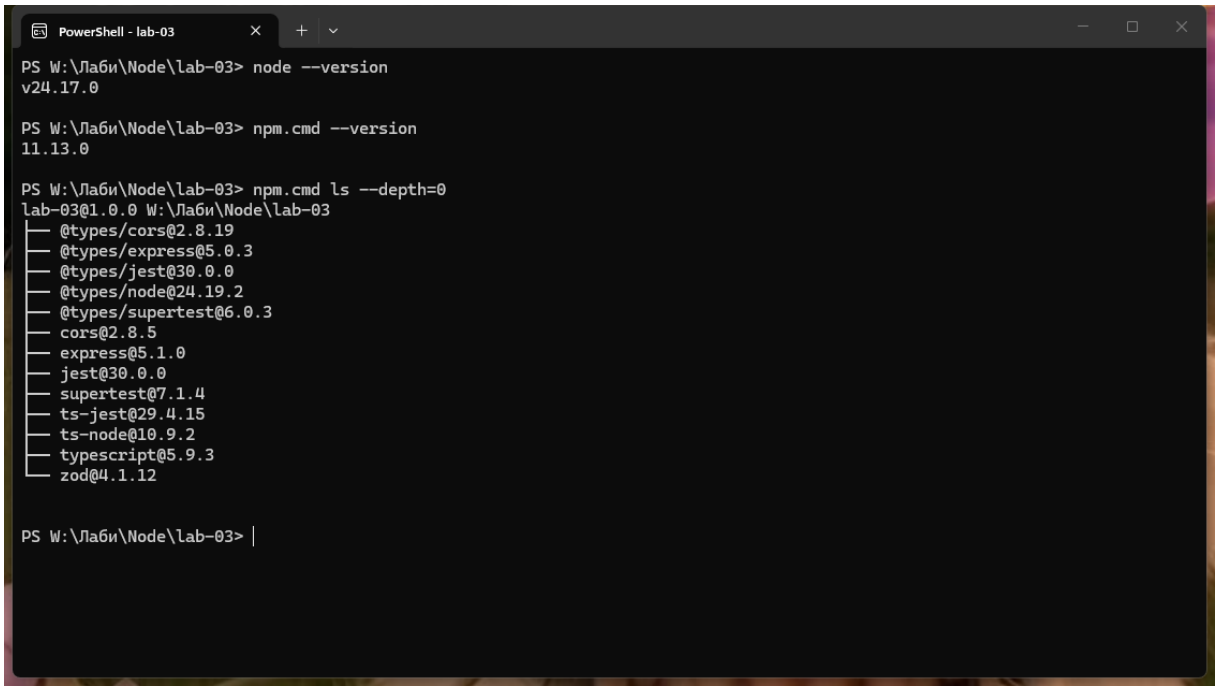


Рис. 1. Встановлені версії Node.js, npm і пакети проєкту

Компонент	Версія
Node.js	24.17.0
npm	11.13.0
Express	5.1.0
CORS	2.8.5
Zod	4.1.12
TypeScript	5.9.3
Jest	30.0.0
Supertest	7.1.4

2 Структура проєкту і сценарії npm

Структуру поділено за відповідальністю: schemas містить правила даних, storage — операції над Map, routes — HTTP-маршрути, middleware — перевірку та обробку помилок. app.ts налаштовує Express, а server.ts запускає прослуховування порту. Тести розташовано окремо в tests. Зібрані JavaScript-файли записуються в dist.

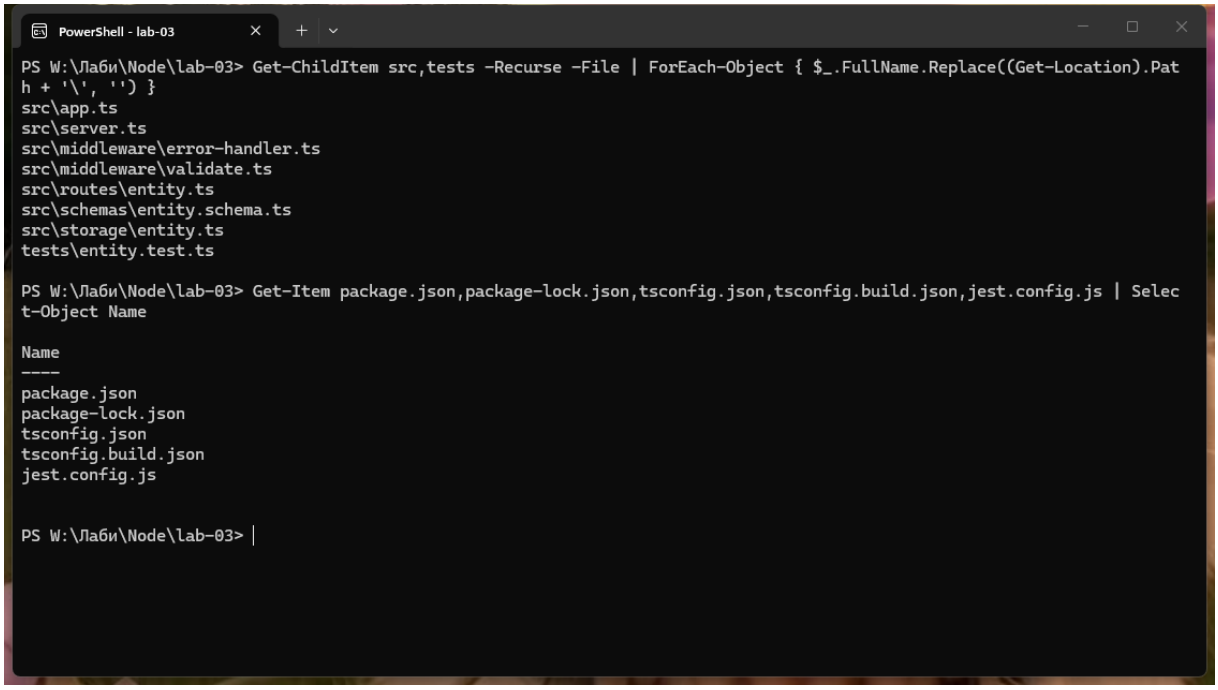


Рис. 2. Файли вихідного коду, тестів і конфігурації

Скрипт dev запускає TypeScript без окремої збірки. Скрипт build компілює тільки src, typecheck перевіряє також тести, test виконує Jest, test:coverage вимірює покриття. Залежності виконання відокремлено від інструментів розробки.

Лістинг 1 — package.json

```
{
  "name": "lab-03",
  "version": "1.0.0",
  "private": true,
  "description": "Лабораторна робота 3. REST API рецептів з Express і Supertest",
  "author": "Сокирко Владислав Володимирович, ІПЗ-23-4",
  "type": "commonjs",
  "scripts": {
    "dev": "ts-node src/server.ts",
    "build": "tsc -p tsconfig.build.json",
    "typecheck": "tsc --noEmit",
    "test": "jest --runInBand",
    "test:coverage": "jest --coverage --runInBand"
  },
  "dependencies": {
    "cors": "2.8.5",
    "express": "5.1.0",
    "zod": "4.1.12"
  },
  "devDependencies": {
    "@types/cors": "2.8.19",
    "@types/express": "5.0.3",
    "@types/jest": "30.0.0",
    "@types/node": "24.19.2",
    "@types/supertest": "6.0.3",
    "jest": "30.0.0",
    "supertest": "7.1.4",
    "ts-jest": "29.4.15",
    "ts-node": "10.9.2",
    "typescript": "5.9.3"
  }
}
```

Лістинг 2 — tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "CommonJS",
    "moduleResolution": "Node",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "esModuleInterop": true,
    "noEmitOnError": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true,
    "types": ["node", "jest"]
  },
  "include": ["src/**/*.ts", "tests/**/*.ts"],
  "exclude": ["node_modules", "dist", "coverage"]
}
```

Лістинг 3 — tsconfig.build.json

```
{
  "extends": "./tsconfig.json",
  "compilerOptions": {
    "rootDir": "src",
    "outDir": "dist"
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist", "coverage", "tests"]
```

```
}
```

Лістинг 4 — jest.config.js

```
module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'node',
  reporters: ['default'],
  testMatch: ['**/tests/**/*.test.ts'],
  collectCoverageFrom: ['src/**/*.ts', '!src/server.ts'],
  coverageDirectory: 'coverage',
  coverageReporters: ['text', 'html', 'lcov', 'json-summary']
};
```

3 Схеми даних і типізація

createSchema перевіряє назву від 1 до 100 символів, необов'язковий опис до 500 символів, цілий час приготування від 1 до 1440 хвилин та один із трьох рівнів складності. updateSchema робить усі поля необов'язковими для часткового PATCH. Типи вхідних даних і фільтрів виведено з Zod через z.infer; збережена сутність додатково має UUID та часові мітки. Параметри запиту також перевіряються схемою Zod: maxMinutes перетворюється з рядка URL на число.

Лістинг 5 — src/schemas/entity.schema.ts

```
import { z } from 'zod';


export const createSchema = z.object({
  title: z.string().min(1).max(100),
  description: z.string().max(500).optional(),
  prepMinutes: z.number().int().min(1).max(1440),
  difficulty: z.enum(['easy', 'medium', 'hard'])
});


export const updateSchema = createSchema.partial();


export const listQuerySchema = z.object({
  difficulty: createSchema.shape.difficulty.optional(),
  maxMinutes: z.coerce.number().int().min(1).max(1440).optional()
});


export type RecipeInput = z.infer<typeof createSchema>;
export type RecipeUpdate = z.infer<typeof updateSchema>;
export type RecipeFilters = z.infer<typeof listQuerySchema>;
export type Entity = RecipeInput & {
  id: string;
  createdAt: string;
  updatedAt: string;
};
```

4 Сховище Map і логіка фільтрації

Map<string, Entity> зберігає рецепти за їхнім id. Метод listEntities застосовує обидва необов'язкові фільтри на рівні сховища; за наявності обох параметрів запис має відповідати обом умовам. Створення генерує UUID через crypto.randomUUID і дві дати, оновлення зберігає попередні поля та змінює updatedAt, видалення повертає факт успіху. resetEntities очищає Map перед кожним тестом.

Лістинг 6 — src/storage/entity.ts

```
import { randomUUID } from 'node:crypto';
import { Entity, RecipeFilters, RecipeInput, RecipeUpdate } from '../schemas/entity.schema';


const recipes = new Map<string, Entity>();


export function listEntities(filters: RecipeFilters = {}): Entity[] {
  return Array.from(recipes.values()).filter((recipe) =>
    (filters.difficulty === undefined || recipe.difficulty === filters.difficulty) &&
    (filters.maxMinutes === undefined || recipe.prepMinutes <= filters.maxMinutes)
  );
}


export function getEntity(id: string): Entity | undefined {
  return recipes.get(id);
}


export function createEntity(input: RecipeInput): Entity {
  const now = new Date().toISOString();
  const recipe: Entity = { ...input, id: randomUUID(), createdAt: now, updatedAt: now };
  recipes.set(recipe.id, recipe);
  return recipe;
}


export function updateEntity(id: string, input: RecipeUpdate): Entity | undefined {
  const current = recipes.get(id);
  if (current === undefined) return undefined;
  const updated: Entity = { ...current, ...input, updatedAt: new Date().toISOString() };
  recipes.set(id, updated);
  return updated;
}


export function deleteEntity(id: string): boolean {
  return recipes.delete(id);
}


export function resetEntities(): void {
  recipes.clear();
}
```

5 Middleware і обробка помилок

Middleware validate застосовує відповідну схему до тіла POST або PATCH і передає помилку наступному обробнику. errorHandler перетворює ZodError на 400

зі списком проблемних полів, HttpError на його HTTP-статус, синтаксично неправильний JSON на 400, а непередбачену помилку на 500. У відповіді 500 не розкриваються внутрішні подробиці сервера.

Лістинг 7 — src/middleware/validate.ts

```
import { NextFunction, Request, Response } from 'express';
import { z } from 'zod';


export function validate(schema: z.ZodType) {
  return (req: Request, _res: Response, next: NextFunction): void => {
    try {
      req.body = schema.parse(req.body);
      next();
    } catch (error) {
      next(error);
    }
  };
}
```

Лістинг 8 — src/middleware/error-handler.ts

```
import { ErrorRequestHandler } from 'express';
import { ZodError } from 'zod';


export class HttpError extends Error {
  constructor(public readonly status: number, message: string) {
    super(message);
  }
}


export const errorHandler: ErrorRequestHandler = (error: unknown, _req, res, _next) => {
  if (error instanceof ZodError) {
    res.status(400).json({
      error: 'VALIDATION_ERROR',
      issues: error.issues.map((issue) => ({ path: issue.path.join('.'), message:
issue.message }))
    });
    return;
  }
  if (error instanceof HttpError) {
    res.status(error.status).json({ error: 'HTTP_ERROR', message: error.message });
    return;
  }
  if (error instanceof SyntaxError && 'status' in error && error.status === 400) {
    res.status(400).json({ error: 'INVALID_JSON', message: 'Тіло запиту не є коректним JSON'
});
    return;
  }
  res.status(500).json({ error: 'INTERNAL_ERROR', message: 'Помилка сервера' });
};
```

6 REST-маршрути та запуск застосунку

Метод	Шлях	Статус успіху	Дія
GET	/api/recipes	200	Список; difficulty і maxMinutes
GET	/api/recipes/quick	200	Рецепти не довше 30 хв

Метод	Шлях	Статус успіху	Дія
GET	/api/recipes/:id	200	Один рецепт
POST	/api/recipes	201	Створення; заголовок Location
PATCH	/api/recipes/:id	200	Часткове оновлення
DELETE	/api/recipes/:id	204	Видалення без тіла

Маршрут /quick оголошено перед /:id, щоб слово quick не сприймалося як ідентифікатор. Відсутній рецепт або невідомий шлях дає 404. Послідовність у app.ts: CORS, JSON-парсер, маршрути, 404-middleware, errorHandler. app.ts не відкриває порт, тому Supertest напряму перевіряє app; server.ts потрібен для звичайного запуску.

Лістинг 9 — src/routes/entity.ts

```
import { Router } from 'express';
import { createSchema, listQuerySchema, updateSchema } from '../schemas/entity.schema';
import { createEntity, deleteEntity, getEntity, listEntities, updateEntity } from
'../storage/entity';
import { HttpError } from '../middleware/error-handler';
import { validate } from '../middleware/validate';


export const entityRouter = Router();


entityRouter.get('/', (req, res) => {
  const filters = listQuerySchema.parse(req.query);
  res.json(listEntities(filters));
});


entityRouter.get('/quick', (_req, res) => {
  res.json(listEntities({ maxMinutes: 30 }));
});


entityRouter.get('/:id', (req, res) => {
  const recipe = getEntity(req.params.id);
  if (recipe === undefined) throw new HttpError(404, 'Рецепт не знайдено');
  res.json(recipe);
});


entityRouter.post('/', validate(createSchema), (req, res) => {
  const recipe = createEntity(req.body);
  res.location(`/api/recipes/${recipe.id}`).status(201).json(recipe);
});


entityRouter.patch('/:id', validate(updateSchema), (req, res) => {
  const recipe = updateEntity(req.params.id as string, req.body);
  if (recipe === undefined) throw new HttpError(404, 'Рецепт не знайдено');
  res.json(recipe);
});


entityRouter.delete('/:id', (req, res) => {
  if (!deleteEntity(req.params.id)) throw new HttpError(404, 'Рецепт не знайдено');
  res.status(204).send();
```

```
});
```

Лістинг 10 — src/app.ts

```
import cors from 'cors';
import express from 'express';
import { entityRouter } from './routes/entity';
import { errorHandler, HttpError } from './middleware/error-handler';


export const app = express();


app.use(cors());
app.use(express.json());
app.use('/api/recipes', entityRouter);
app.use((_req, _res, next) => next(new HttpError(404, 'Маршрут не знайдено')));
app.use(errorHandler);
```

Лістинг 11 — src/server.ts

```
import { app } from './app';


const port = Number(process.env.PORT ?? 3000);


app.listen(port, () => {
  process.stdout.write(`Recipe API: http://localhost:${port}/api/recipes\n`);
});
```

7 Перевірка типів і збірка

Команди npm run typecheck та npm run build завершилися без діагностик. Після збірки в dist з’явилися JavaScript-файли для всіх модулів src. Це перевіряє узгодженість TypeScript-типів і можливість звичайного запуску Node.js.

```
npm.cmd run typecheck
npm.cmd run build
```

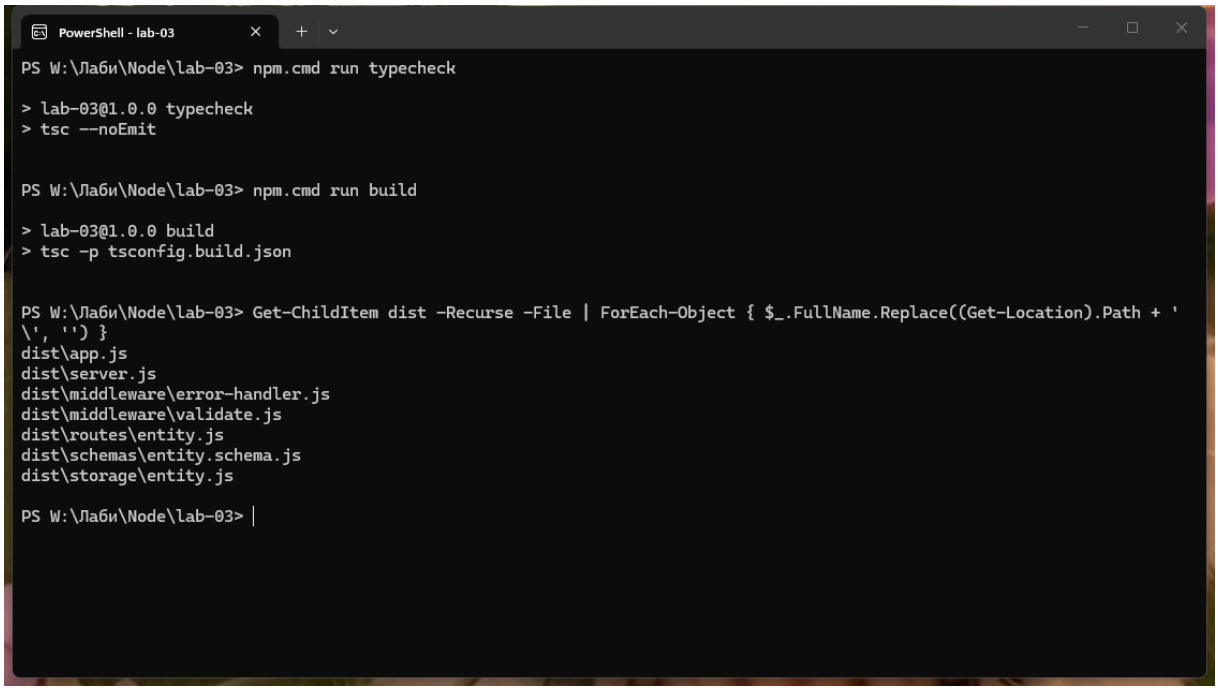


Рис. 3. Успішна перевірка типів і результат компіляції

8 Інтеграційні тести Supertest

Supertest надсилає HTTP-запити до app без ручного запуску сервера. beforeEach викликає resetEntities, отже результати тестів не залежать від порядку. Перевірено створення й граничні значення схеми, заголовок Location, UUID і дати, читання за id, 404, фільтри окремо та разом, /quick, часткове оновлення, незмінність запису після невдалого PATCH, видалення з порожнім тілом, некоректний JSON і CORS preflight.

Лістинг 12 — tests/entity.test.ts

```
import request from 'supertest';
import { app } from '../src/app';
import { resetEntities } from '../src/storage/entity';


const base = '/api/recipes';
const omelette = { title: 'Омлет', description: 'Швидкий сніданок', prepMinutes: 12,
difficulty: 'easy' };
const soup = { title: 'Суп', prepMinutes: 45, difficulty: 'medium' };
const roast = { title: 'Печеня', prepMinutes: 90, difficulty: 'hard' };


beforeEach(() => resetEntities());


async function addRecipe(recipe: object) {
  return request(app).post(base).send(recipe);
}


describe('POST /api/recipes', () => {
  test('створює рецепт зі статусом 201, UUID, датами та Location', async () => {
    const response = await addRecipe(omelette);
    expect(response.status).toBe(201);
    expect(response.headers['content-type']).toMatch(/json/);
    expect(response.headers.location).toBe(`${base}/${response.body.id}`);
```

```
    expect(response.body).toMatchObject(omelette);
    expect(response.body.id).toMatch(/^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-
f]{3}-[0-9a-f]{12}$/i);
    expect(new Date(response.body.createdAt).toISOString()).toBe(response.body.createdAt);
    expect(response.body.updatedAt).toBe(response.body.createdAt);
  });


  test('приймає рецепт без необов’язкового опису', async () => {
    const response = await addRecipe(soup);
    expect(response.status).toBe(201);
    expect(response.body.description).toBeUndefined();
  });


  test('відхиляє порожню назву', async () => {
    const response = await addRecipe({ ...omelette, title: '' });
    expect(response.status).toBe(400);
    expect(response.body).toMatchObject({ error: 'VALIDATION_ERROR' });
    expect(response.body.issues).toEqual(expect.arrayContaining([expect.objectContaining({
path: 'title' })]));
  });


  test('відхиляє назву понад 100 символів', async () => {
    const response = await addRecipe({ ...omelette, title: 'a'.repeat(101) });
    expect(response.status).toBe(400);
  });


  test('приймає граничну назву зі 100 символів', async () => {
    const response = await addRecipe({ ...omelette, title: 'a'.repeat(100) });
    expect(response.status).toBe(201);
  });


  test('відхиляє опис понад 500 символів', async () => {
    const response = await addRecipe({ ...omelette, description: 'a'.repeat(501) });
    expect(response.status).toBe(400);
  });


  test('відхиляє відсутній час приготування', async () => {
    const { prepMinutes: _omitted, ...recipe } = omelette;
    const response = await addRecipe(recipe);
    expect(response.status).toBe(400);
  });


  test('відхиляє дробовий час приготування', async () => {
    const response = await addRecipe({ ...omelette, prepMinutes: 12.5 });
    expect(response.status).toBe(400);
  });


  test('відхиляє недопустиму складність', async () => {
    const response = await addRecipe({ ...omelette, difficulty: 'extreme' });
    expect(response.status).toBe(400);
  });


  test('відхиляє некоректний JSON', async () => {
    const response = await request(app).post(base).set('Content-Type',
'application/json').send('{broken');
    expect(response.status).toBe(400);
    expect(response.body.error).toBe('INVALID_JSON');
  });
});
```

```
describe('GET /api/recipes', () => {
  test('повертає порожній список на початку тесту', async () => {
    const response = await request(app).get(base);
    expect(response.status).toBe(200);
    expect(response.body).toEqual([]);
  });


  test('повертає всі створені рецепти', async () => {
    await addRecipe(omelette);
    await addRecipe(soup);
    const response = await request(app).get(base);
    expect(response.status).toBe(200);
    expect(response.body.map((recipe: { title: string }) => recipe.title)).toEqual(['Омлет',
'Суп']);
  });


  test('фільтрує за складністю', async () => {
    await addRecipe(omelette);
    await addRecipe(soup);
    const response = await request(app).get(base).query({ difficulty: 'easy' });
    expect(response.status).toBe(200);
    expect(response.body.map((recipe: { title: string }) =>
recipe.title)).toEqual(['Омлет']);
  });


  test('фільтрує за максимальним часом включно з межею', async () => {
    await addRecipe(omelette);
    await addRecipe(soup);
    const response = await request(app).get(base).query({ maxMinutes: 12 });
    expect(response.status).toBe(200);
    expect(response.body.map((recipe: { title: string }) =>
recipe.title)).toEqual(['Омлет']);
  });


  test('комбінує обидва фільтри', async () => {
    await addRecipe(omelette);
    await addRecipe({ ...soup, difficulty: 'easy' });
    await addRecipe(roast);
    const response = await request(app).get(base).query({ difficulty: 'easy', maxMinutes: 20
});
    expect(response.status).toBe(200);
    expect(response.body.map((recipe: { title: string }) =>
recipe.title)).toEqual(['Омлет']);
  });


  test('повертає порожній масив, якщо фільтри не дали збігів', async () => {
    await addRecipe(omelette);
    const response = await request(app).get(base).query({ difficulty: 'hard', maxMinutes: 5
});
    expect(response.status).toBe(200);
    expect(response.body).toEqual([]);
  });


  test('відхиляє некоректну складність у query', async () => {
    const response = await request(app).get(base).query({ difficulty: 'extreme' });
    expect(response.status).toBe(400);
    expect(response.body.error).toBe('VALIDATION_ERROR');
  });


  test('відхиляє недодатний час у query', async () => {
    const response = await request(app).get(base).query({ maxMinutes: 0 });
    expect(response.status).toBe(400);
```

```
  });
});


describe('GET /api/recipes/quick', () => {
  test('повертає лише рецепти до 30 хвилин', async () => {
    await addRecipe(omelette);
    await addRecipe({ ...soup, prepMinutes: 30 });
    await addRecipe(roast);
    const response = await request(app).get(`${base}/quick`);
    expect(response.status).toBe(200);
    expect(response.body.map((recipe: { title: string }) => recipe.title)).toEqual(['Омлет',
'Суп']);
  });


  test('повертає порожній список без швидких рецептів', async () => {
    await addRecipe(roast);
    const response = await request(app).get(`${base}/quick`);
    expect(response.body).toEqual([]);
  });
});


describe('GET /api/recipes/:id', () => {
  test('повертає створений рецепт', async () => {
    const created = await addRecipe(omelette);
    const response = await request(app).get(`${base}/${created.body.id}`);
    expect(response.status).toBe(200);
    expect(response.body).toEqual(created.body);
  });


  test('повертає 404 для відсутнього id', async () => {
    const response = await request(app).get(`${base}/absent`);
    expect(response.status).toBe(404);
    expect(response.body.error).toBe('HTTP_ERROR');
  });
});


describe('PATCH /api/recipes/:id', () => {
  test('змінює лише передані поля', async () => {
    const created = await addRecipe(omelette);
    const response = await request(app).patch(`${base}/${created.body.id}`).send({ title:
'Омлет із сиром', prepMinutes: 15 });
    expect(response.status).toBe(200);
    expect(response.body).toMatchObject({ title: 'Омлет із сиром', prepMinutes: 15,
difficulty: 'easy', description: omelette.description });
    expect(response.body.createdAt).toBe(created.body.createdAt);
    expect(new Date(response.body.updatedAt).getTime()).toBeGreaterThanOrEqual(new
Date(created.body.updatedAt).getTime());
  });


  test('повертає 404 для відсутнього id', async () => {
    const response = await request(app).patch(`${base}/absent`).send({ title: 'Інший' });
    expect(response.status).toBe(404);
  });


  test('відхиляє невалідний час та не змінює запис', async () => {
    const created = await addRecipe(omelette);
    const response = await request(app).patch(`${base}/${created.body.id}`).send({
prepMinutes: -1 });
    expect(response.status).toBe(400);
    const unchanged = await request(app).get(`${base}/${created.body.id}`);
    expect(unchanged.body.prepMinutes).toBe(12);
  });
```

```
});


describe('DELETE /api/recipes/:id', () => {
  test('видаляє запис зі статусом 204 і порожнім тілом', async () => {
    const created = await addRecipe(omelette);
    const response = await request(app).delete(`${base}/${created.body.id}`);
    expect(response.status).toBe(204);
    expect(response.text).toBe('');
    const absent = await request(app).get(`${base}/${created.body.id}`);
    expect(absent.status).toBe(404);
  });


  test('повертає 404 при повторному видаленні', async () => {
    const created = await addRecipe(omelette);
    await request(app).delete(`${base}/${created.body.id}`);
    const response = await request(app).delete(`${base}/${created.body.id}`);
    expect(response.status).toBe(404);
  });
});


describe('налаштування застосунку', () => {
  test('повертає 404 для невідомого маршруту', async () => {
    const response = await request(app).get('/api/unknown');
    expect(response.status).toBe(404);
  });


  test('відповідає на CORS preflight', async () => {
    const response = await request(app).options(base).set('Origin',
'http://example.com').set('Access-Control-Request-Method', 'POST');
    expect(response.status).toBe(204);
    expect(response.headers['access-control-allow-origin']).toBe('*');
  });
});
```

Звичайний запуск Jest дав 29 успішних перевірок в одному тестовому наборі. Запуск із покриттям підтвердив той самий результат і показав розподіл за файлами. У показник покриття включено модулі src, крім server.ts: він лише запускає мережевий процес, тоді як тести викликають app напряму.

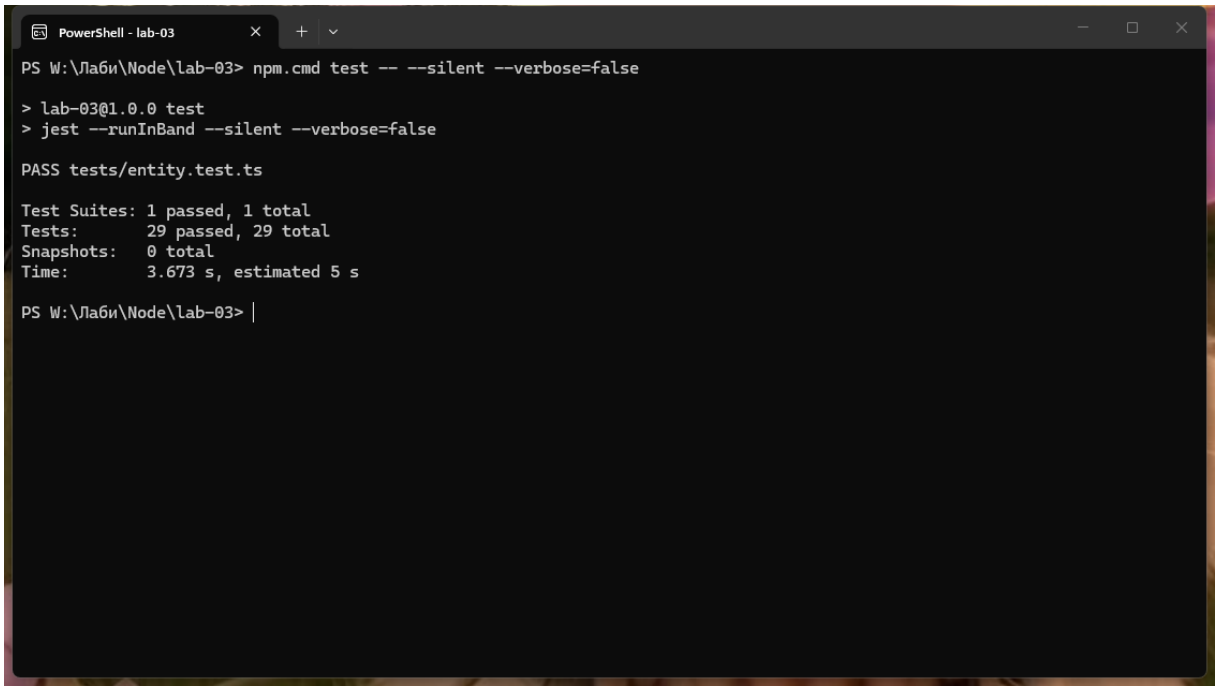


Рис. 4. Результат виконання 29 інтеграційних тестів

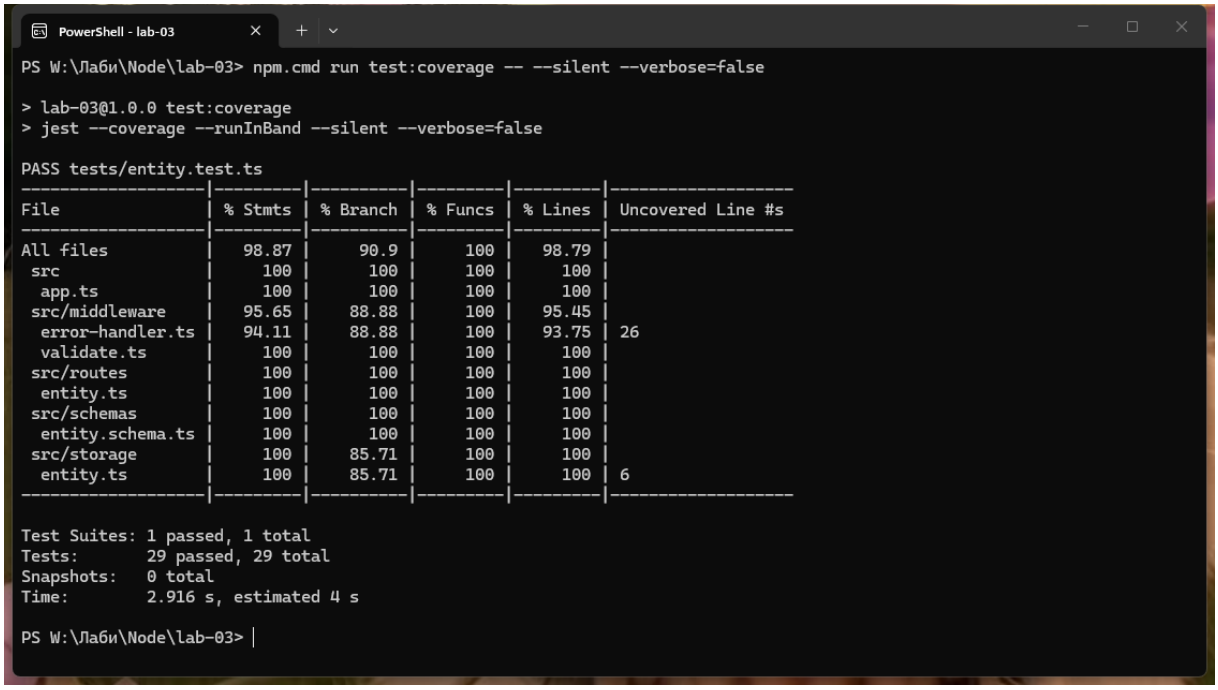


Рис. 5. Покриття коду та підсумок тестів

Метрика	**Покрито**	**Усього**	**Відсоток**
Інструкції	88	89	98.87%
Гілки	20	22	90.9%
Функції	19	19	100%
Рядки	82	83	98.79%

Непокритими залишаються захисна гілка невідомої внутрішньої помилки та частина гілок значень за замовчуванням. Це не впливає на виконання вимоги про мінімум 12 інтеграційних тестів, але показує межі цього набору перевірок.

9 Перевірка запитів у PowerShell

Після компіляції запущено dist/server.js на локальному порту 3000. PowerShell-команди нижче виконані у справжньому вікні консолі. Початковий GET повернув 200 і порожній масив, що підтверджує відсутність початкових даних у Map.

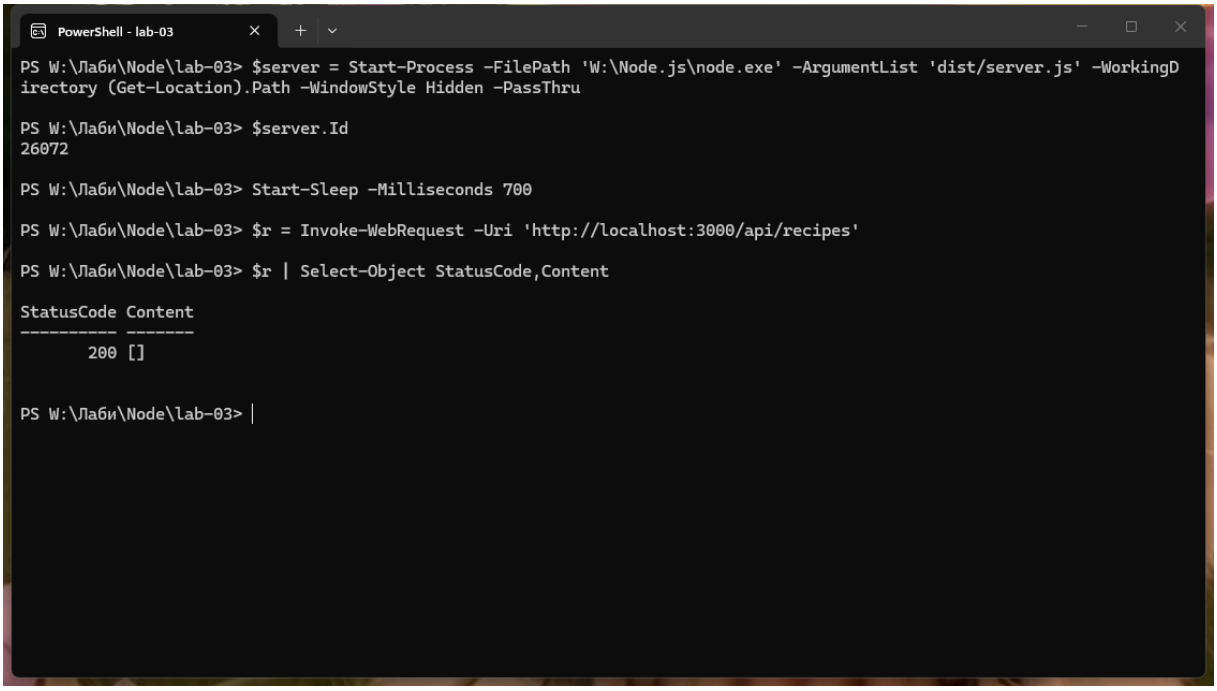


Рис. 6. Запуск локального сервера та порожній список рецептів

POST з коректним JSON створив рецепт «Омлет». У відповіді отримано 201, Location із шляхом нового ресурсу, UUID, дані рецепта та createdAt/updatedAt. Дати генерує сервер.

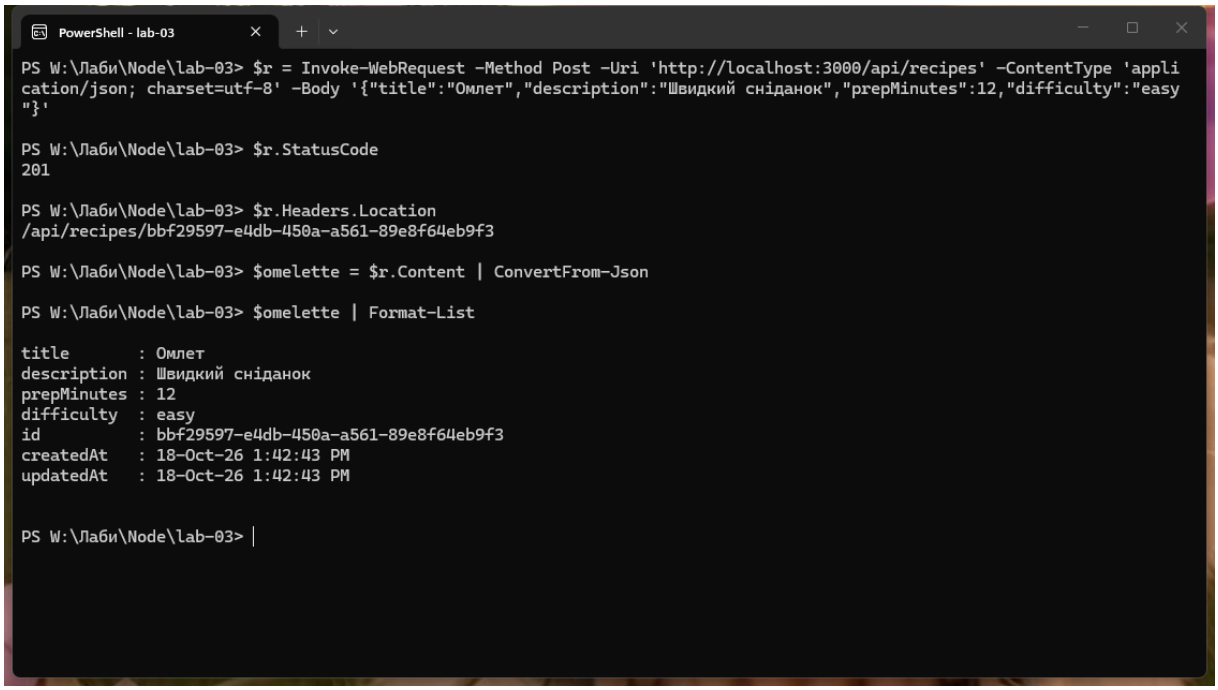


Рис. 7. Створення рецепта: статус 201, Location і серверні поля

Після додавання зразків «Суп» і «Печеня» GET списку повернув усі три записи. Кожен рядок у консолі показує назву, час і складність — поля, потрібні для подальшої перевірки фільтрів.

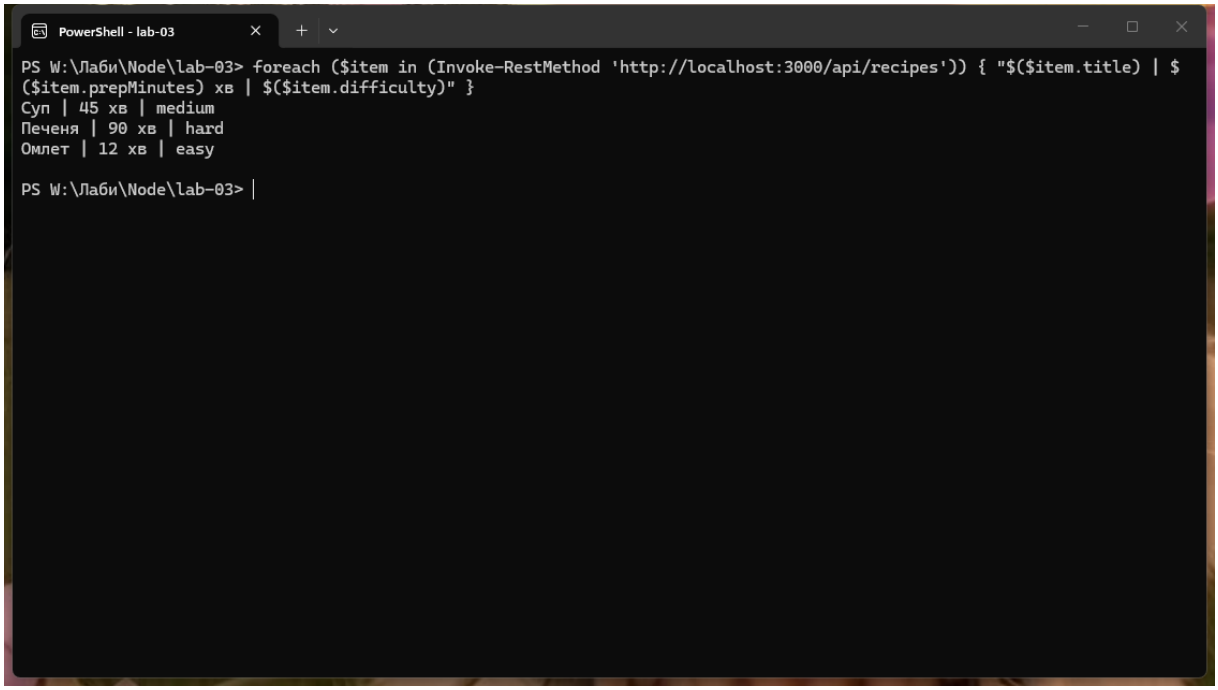


Рис. 8. Три рецепти у відповіді GET /api/recipes

difficulty=easy залишає «Омлет»; maxMinutes=45 залишає «Суп» і «Омлет»; обидва параметри difficulty=medium&maxMinutes=45 разом залишають лише «Суп». Це демонструє роботу незалежних і комбінованих умов у сховищі.

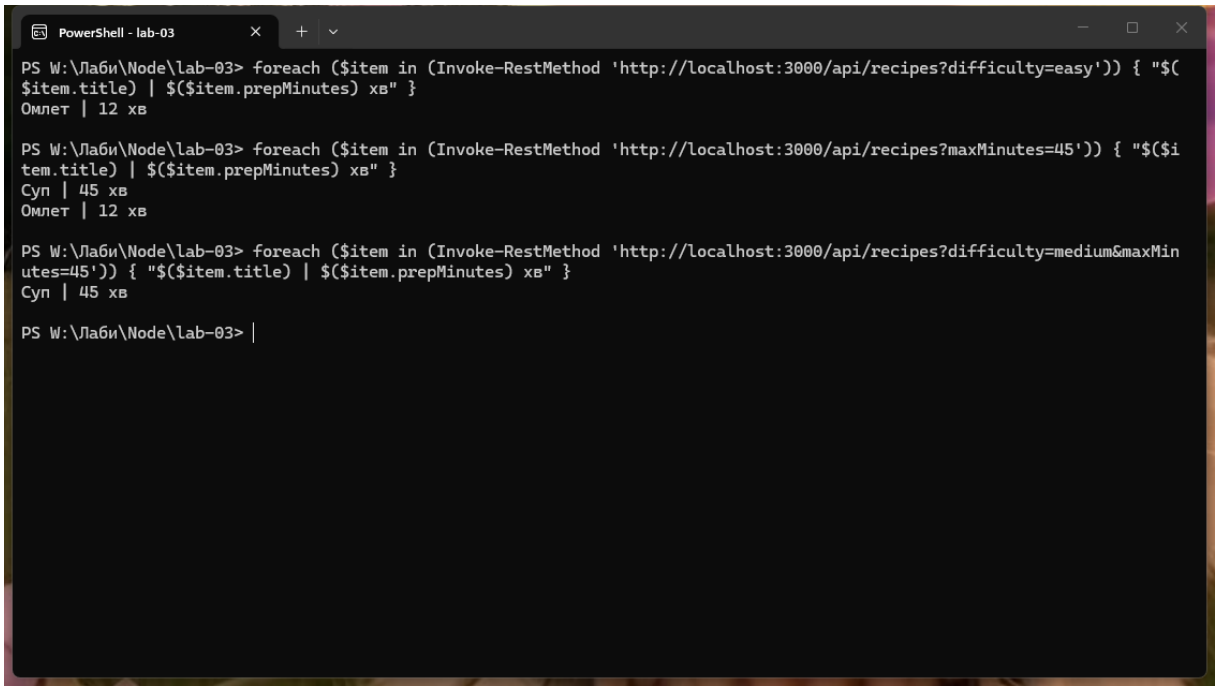


Рис. 9. Два query-фільтри окремо та в комбінації

Додатковий шлях /api/recipes/quick повертає лише рецепти, що готуються не довше 30 хвилин. У поточних даних це «Омлет» із тривалістю 12 хвилин.

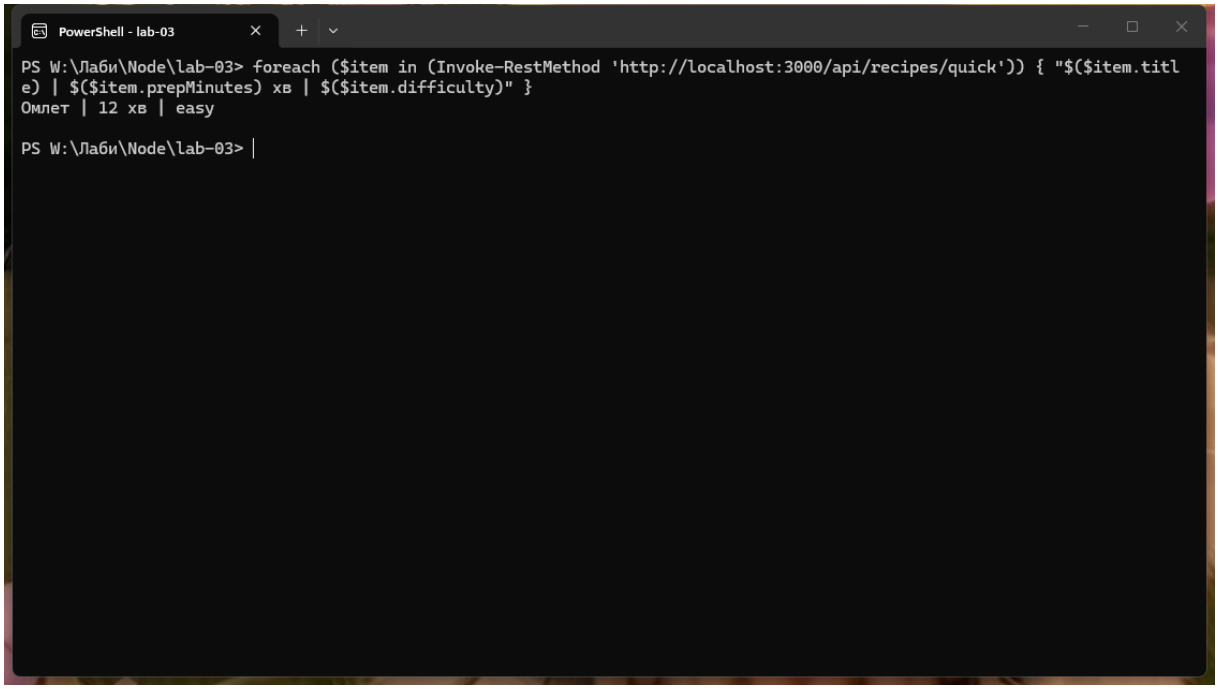


Рис. 10. Предметний маршрут швидких рецептів

GET за UUID повернув вибраний рецепт. PATCH змінив час з 12 на 18 хвилин зі статусом 200, а інші поля залишив без змін. У тестах також перевірено незмінність createdAt і коректність updatedAt.

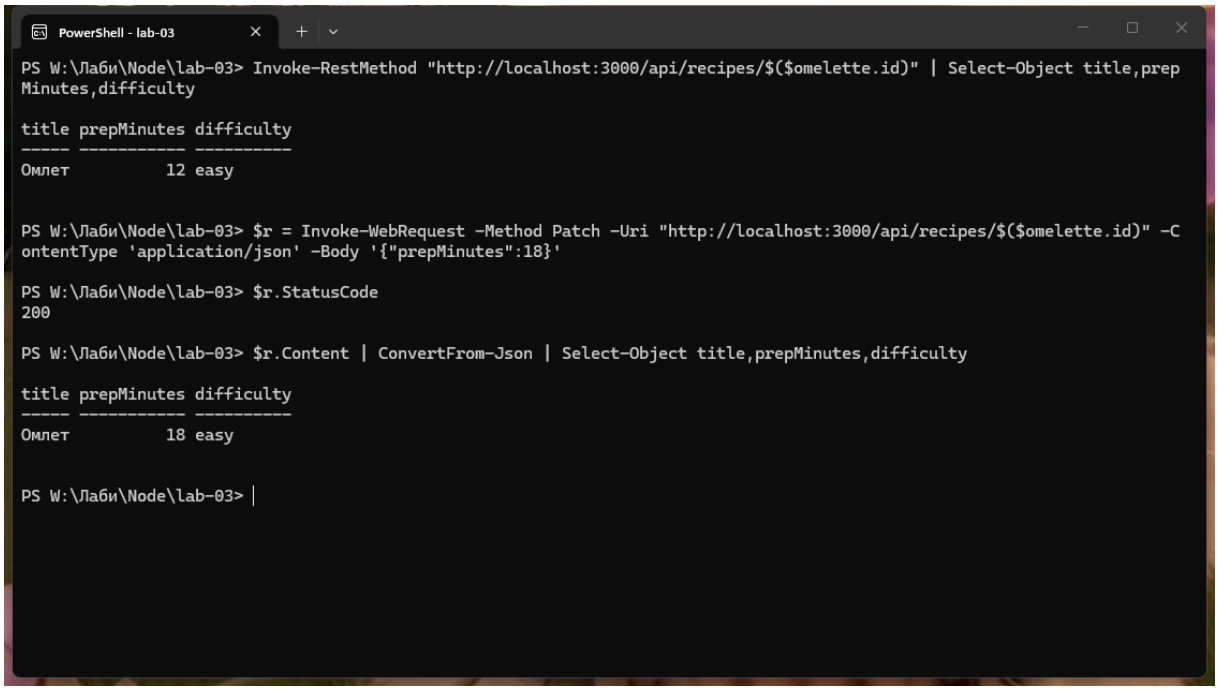


Рис. 11. Читання за id та часткове оновлення рецепта

Некоректне тіло POST із порожньою назвою, часом 0 та недопустимою складністю повертає 400 і перелік проблем. Запит до відсутнього id повертає 404. Обидві помилки мають JSON-відповідь із повідомленням для клієнта.

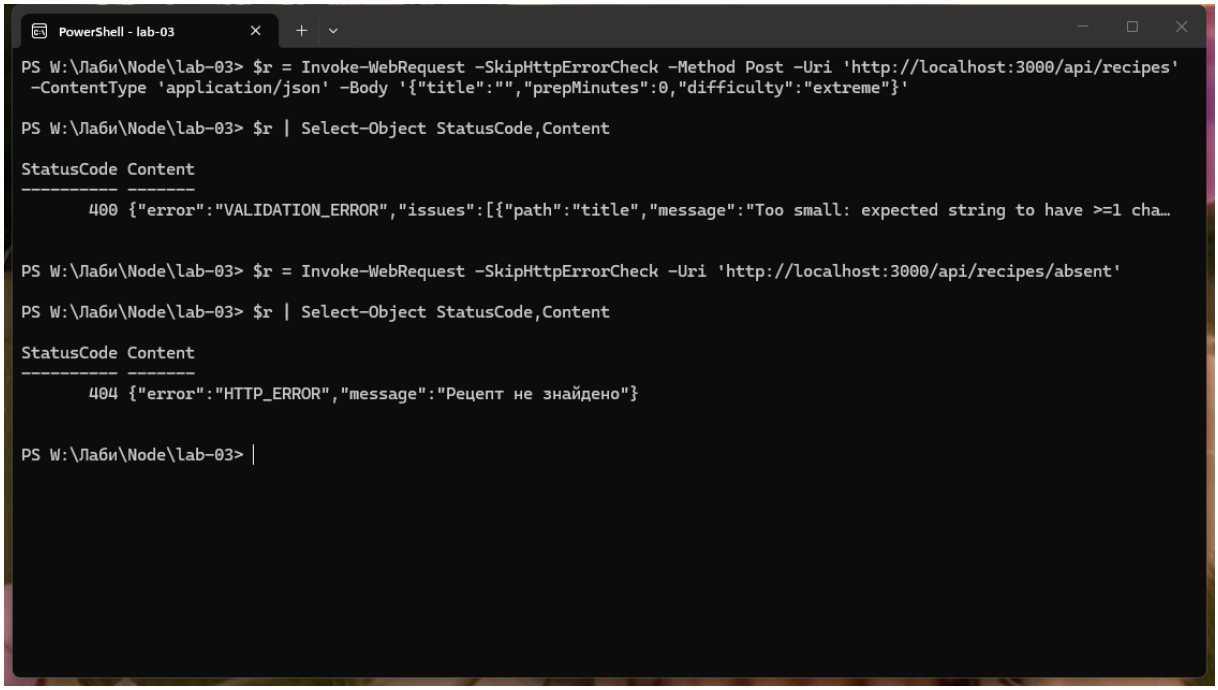


Рис. 12. Валідація запиту 400 і відсутній ресурс 404

DELETE вибраного рецепта повертає 204 і не містить тіла. Повторний GET того самого id дає 404, тобто запис справді видалено зі сховища.

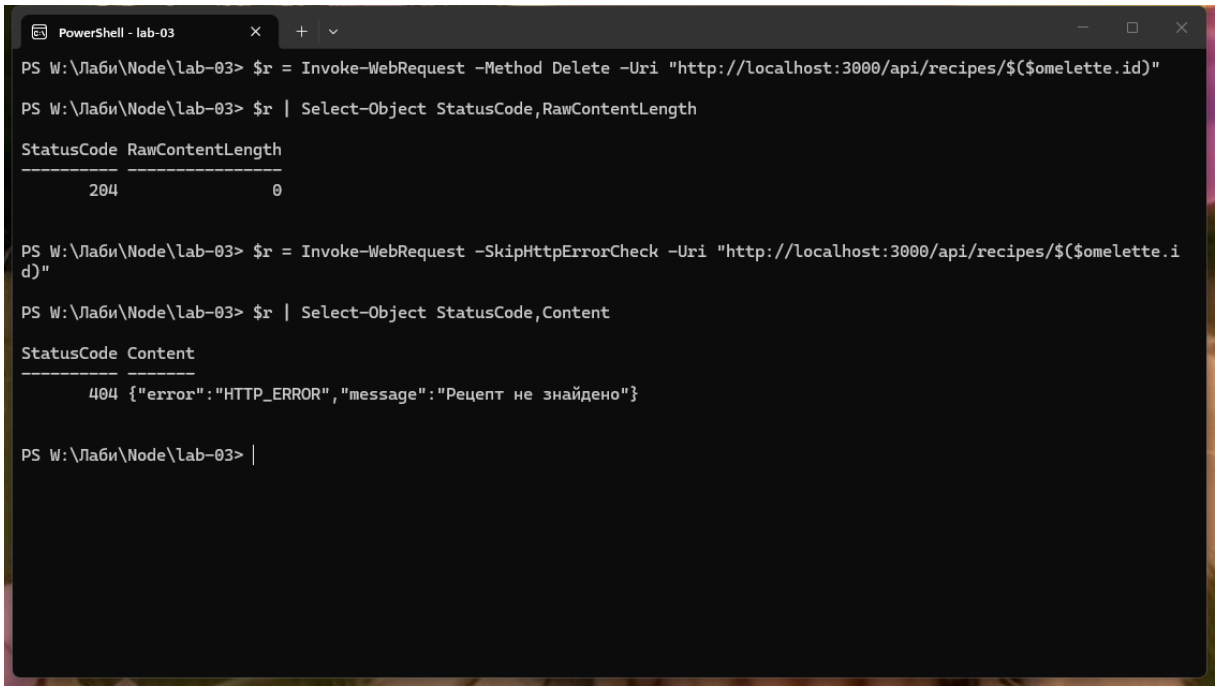


Рис. 13. Видалення рецепта та перевірка його відсутності

Висновок:

Розроблено TypeScript REST API рецептів на Express.js. Zod перевіряє тіла запитів і параметри фільтрації, Map забезпечує операції CRUD, а централізоване middleware формує відповіді на помилки. Розділення app.ts і server.ts дозволило перевіряти HTTP-поведінку без окремого процесу сервера.

Реалізовано два предметні фільтри з можливістю комбінування та маршрут /quick. Усі 29 інтеграційних тестів пройшли; типи та збірка перевірені окремо. Виконання основних операцій і статус-кодів підтверджене скріншотами реального PowerShell. Дані зберігаються тільки під час роботи процесу, що відповідає умові in-memory storage.